Busca Cega em Python

Objetivo: implementar um algoritmo de **Busca em Largura (BFS)** para encontrar um caminho entre dois pontos sem utilizar heurística.

Situação-problema

Você está desenvolvendo um sistema para um robô que precisa atravessar um labirinto. O robô começa na posição `S` e precisa chegar até `G`.

O mapa é:

```
mapa = [
    ['S', '0', '1', '0', '0'],
    ['1', '0', '1', '0', '1'],
    ['0', '0', '0', '0', '1'],
    ['0', '1', '1', '0', '0'],
    ['1', '0', '0', '0', 'G']
]
```

Onde:

- `S` = posição inicial;
- `G` = objetivo;
- `0` = caminho livre;
- `1` = parede.

Tarefa

Crie um programa em Python que utilize **Busca em Largura (BFS)** para encontrar o caminho de `S` até `G`.

O programa deverá:

1. Localizar automaticamente `S` e `G`.
2. Criar uma fila para armazenar as posições que serão exploradas.
3. Visitar as posições nas quatro direções possíveis.
4. Não atravessar paredes.
5. Não visitar novamente uma posição já explorada.
6. Registrar o caminho percorrido.
7. Exibir o caminho encontrado.
8. Informar quantas posições foram exploradas.

Estrutura inicial

Complete o código abaixo:

```
from collections import deque

mapa = [
    ['S', '0', '1', '0', '0'],
    ['1', '0', '1', '0', '1'],
    ['0', '0', '0', '0', '1'],
    ['0', '1', '1', '0', '0'],
    ['1', '0', '0', '0', 'G']
]


def busca_largura(mapa):
    # 1. Encontre S e G


    # 2. Crie a fila


    # 3. Crie o conjunto de visitados


    # 4. Crie uma estrutura para armazenar os caminhos


    # 5. Enquanto houver posições na fila:


        # 6. Retire uma posição da fila


        # 7. Verifique se chegou ao objetivo


        # 8. Explore os quatro vizinhos


        # 9. Adicione os vizinhos válidos à fila


    return None

caminho = busca_largura(mapa)

print("Caminho encontrado:", caminho)
```